
Evidencias del Tercer Avance

AutoPrime · Concesionario de altas prestaciones

Aprendiz: Jose Matias Agudelo Bolivar
Ficha: 3406211 Ambiente: 702 Trimestre: 03
Competencia: React Instructor: Jhan Hader Munoz

Stack: React 19 + Vite -> Node.js + Express -> MySQL (MariaDB)
Seguridad: bcrypt (hash de contraseñas) + JWT (autenticación)

Cada pareja muestra el código y el resultado real que produce.

Registro de clientes con contraseña hasheada

El formulario de registro del segundo avance ya no guarda nada en el navegador: envía los datos al endpoint POST /api/auth/registro. El controlador vuelve a validarlos en el servidor (no confía en el frontend, que se puede saltar con Postman), comprueba que el correo y el documento no estén repetidos y recién ahí cifra la contraseña con bcrypt antes de insertarla.

CAPTURA A · CODIGO

```
backend / src / controllers / auth.controller.js
14 async function registro(req, res, next) {
15   try {
16     const { errores, datos } = validarRegistro(req.body);
17
18     if (Object.keys(errores).length > 0) {
19       return res.status(400).json({
20         ok: false,
21         mensaje: "Revisa los datos enviados.",
22         errores,
23       });
24     }
25
26     if (await usuarioModel.existeCorreo(datos.correo)) {
27       return res.status(409).json({
28         ok: false,
29         mensaje: "Ya existe una cuenta registrada con este correo.",
30         errores: { correo: "Este correo ya esta registrado." },
31       });
32     }
33
34     if (await usuarioModel.existeDocumento(datos.tipoDocumento, datos.numeroDocumento)) {
35       return res.status(409).json({
36         ok: false,
37         mensaje: "Ya existe una cuenta con este documento.",
38         errores: { numeroDocumento: "Este documento ya esta registrado." },
39       });
40     }
41
42     // Hash seguro: bcrypt genera su propia sal en cada llamada.
43     const passwordHash = await bcrypt.hash(datos.password, RONDAS);
44
45     const rolCliente = await usuarioModel.getIdRol("cliente");
46     const usuario = await usuarioModel.crear(datos, passwordHash, rolCliente);
47
48     const token = firmarToken(usuario);
49
50     return res.status(201).json({
51       ok: true,
52       mensaje: "Cuenta creada correctamente.",
53       usuario,
54       token,
55     });
56   } catch (error) {
57     return next(error);
58   }
59 }
```

El código: `bcrypt.hash()` genera el hash con su propia sal. La contraseña original nunca se guarda ni se devuelve en la respuesta.

CAPTURA B · RESULTADO

id	nombre	correo	rol	estado	password_hash
1	Jose Matias	admin@autoprima.com.co	administrador	activo	\$2b\$10\$udDAuy2CsxMbvdh011I/O...
2	Carolina	empleado@autoprima.com.co	empleado	activo	\$2b\$10\$QnCG0rDprNsooY1LEKI840...
3	Andres	cliente@autoprima.com.co	cliente	activo	\$2b\$10\$DDA9Hgf30IpuIUQGfFkk90...
4	Laura	laura1787616233@correo.com	cliente	activo	\$2b\$10\$o1L8GdFQnSHyxVcZ3p58Cu...

El resultado en la base de datos: la columna `password_hash` guarda cadenas de 60 caracteres que empiezan por `$2b$10$` — la firma de bcrypt con 10 rondas. Ninguna contraseña quedó en texto plano.

Autenticacion con JWT y control de roles

Al iniciar sesion el backend firma un JSON Web Token que el frontend envia en la cabecera Authorization de cada peticion. Dos middlewares protegen los endpoints: verificarToken comprueba que el token sea valido y que el usuario siga activo, y permitirRoles restringe la ruta a los roles autorizados.

CAPTURA A · CODIGO

```
backend / src / middlewares / auth.middleware.js
26 async function verificarToken(req, res, next) {
27   const cabecera = req.headers.authorization || "";
28
29   if (!cabecera.startsWith("Bearer ")) {
30     return res.status(401).json({
31       ok: false,
32       mensaje: "Falta el token de autenticacion.",
33     });
34   }
35
36   const token = cabecera.slice(7).trim();
37
38   try {
39     const payload = jwt.verify(token, process.env.JWT_SECRET);
40
41     // Se relee el usuario: si lo inactivaron o borraron, el token deja de servir
42     // aunque todavia no haya expirado.
43     const usuario = await usuarioModel.buscarPorId(payload.id);
44
45     if (!usuario) {
46       return res.status(401).json({ ok: false, mensaje: "El usuario ya no existe." });
47     }
48     if (usuario.estado !== "activo") {
49       return res.status(403).json({ ok: false, mensaje: "Tu cuenta esta inactiva." });
50     }
51
52     req.usuario = usuario;
53     return next();
54   } catch (error) {
55     const expirado = error.name === "TokenExpiredError";
56     return res.status(401).json({
57       ok: false,
58       mensaje: expirado
59         ? "La sesion expiro. Inicia sesion de nuevo."
60         : "Token invalido.",
61     });
62   }
63 }
64
65 /** Uso: router.get("/", verificarToken, permitirRoles("administrador"), handler) */
66 function permitirRoles(...roles) {
67   return (req, res, next) => {
68     if (!req.usuario) {
69       return res.status(401).json({ ok: false, mensaje: "No autenticado." });
70     }
71     if (!roles.includes(req.usuario.rol)) {
72       return res.status(403).json({
73         ok: false,
74         mensaje: "No tienes permisos para esta accion.",
75       });
76     }
77     return next();
78   };
79 }
80
```

El codigo: verificarToken relee el usuario en cada peticion, de modo que si un administrador lo inactiva su token deja de servir aunque todavia no haya expirado.

CAPTURA B · RESULTADO

```
Terminal · respuestas de la API protegida con JWT

$ curl http://localhost:3000/api/usuarios
{
  "ok": false,
  "mensaje": "Falta el token de autenticacion."
}

$ curl .../api/usuarios -H "Authorization: Bearer <token de CLIENTE>"
{
  "ok": false,
  "mensaje": "No tienes permisos para esta accion."
}

$ bash backend/sql/pruebas-api.sh
OK 200 POST /auth/login      login administrador
OK 401 GET /auth/perfil      perfil sin token -> 401
OK 403 GET /usuarios        cliente NO puede listar -> 403
OK 200 GET /usuarios        listar como admin
...
RESULTADO: 51 pruebas OK, 0 fallidas
```

El resultado: sin token responde 401; con token de cliente sobre una ruta de administrador responde 403; con token de admin, 200. La bateria completa son 51 pruebas, todas en verde.

Agendamiento de citas desde el boton Cotizar

Al pulsar Cotizar en la ficha de un vehiculo, la aplicacion redirige a /agendar con el modelo y el servicio en la URL. Alli el cliente escoge una fecha y el frontend consulta las horas libres a la API; las franjas ya reservadas aparecen deshabilitadas. Si el cliente no ha iniciado sesion, se le lleva al login y se le devuelve al formulario.

CAPTURA A · CODIGO

```

143  /* ----- Envio ----- */
144  const enviar = async (evento) => {
145      evento.preventDefault();
146      setError("");
147      setErroresCampo({});
148
149      if (!autenticado) {
150          navegar("/login", { state: { desde: `/agendar${window.location.search}` } });
151          return;
152      }
153
154      if (!hora) {
155          setErroresCampo({ hora: "Selecciona una hora disponible." });
156          return;
157      }
158
159      setEnviando(true);
160      try {
161          const datos = await citasApi.crear({
162              fecha,
163              hora,
164              servicioId: Number(servicioId),
165              productoId: vehiculo?.id ?? null,
166              notas: notas.trim() || undefined,
167          });
168          setConfirmada(datos.cita);
169      } catch (e) {
170          setError(e.message);
171          setErroresCampo(e.errores || {});
172          recargarHoras(); // alguien pudo tomar la franja mientras tanto
173      } finally {
174          setEnviando(false);
175      }
176  };
177
  
```

El codigo: la funcion enviar() del formulario. Si otro cliente tomo la franja mientras tanto, el backend responde 409 y la pantalla recarga las horas disponibles.

CAPTURA B · RESULTADO

id	cliente	servicio	vehiculo	fecha	hora	estado
1	Andres Zapata Molina	Peritaje de 120 puntos	Mercedes-AMG GT	2026-08-26	15:00:00	completada
2	Jose Matias Agudelo Bolivar	Prueba de manejo	NULL	2026-08-24	09:00:00	pendiente
4	Andres Zapata Molina	Prueba de manejo	BMW i8	2026-09-04	15:00:00	pendiente

El resultado: la cita queda guardada en la tabla citas, relacionada con el usuario, el servicio y el vehiculo mediante llaves foraneas.